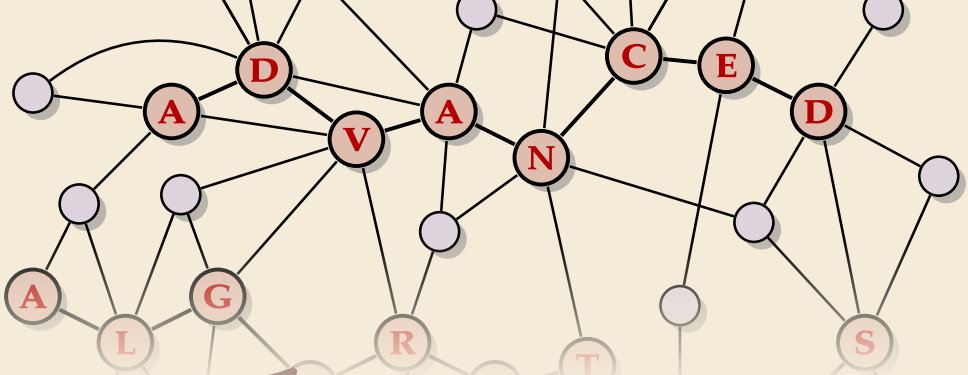


4 Fixed-Parameter Algorithms

Advanced Algorithms · Winter 2026–27

Prof. Dr. Sebastian Wild

CS627 (Winter 2026–27)
Philipps-Universität Marburg
version **2026-10-29 13:33**

Outline

4 Fixed-Parameter Algorithms

4.1 Fixed-Parameter Tractability
4.2 Depth-Bounded Exhaustive Search I
4.3 Problem Kernels
4.4 Depth-Bounded Search II: Planar Indep. Set
4.5 Depth-Bounded Search III: Closest String
4.6 Linear Recurrences & Better Vertex Cover
4.7 Interleaving

Philosophy of FPT

- **Goal:** Principled theory for studying complexity based on two dimensions: input size $n = |x|$ (encoding length) and *some additional parameter* k
  - generalize ideas from $k = MaxInt(x)$
  - investigate influence of k (and n) on running time

Philosophy of FPT

- **Goal:** Principled theory for studying complexity based on two dimensions: input size $n = |x|$ (encoding length) and *some additional parameter* k
  - generalize ideas from $k = MaxInt(x)$
  - investigate influence of k (and n) on running time

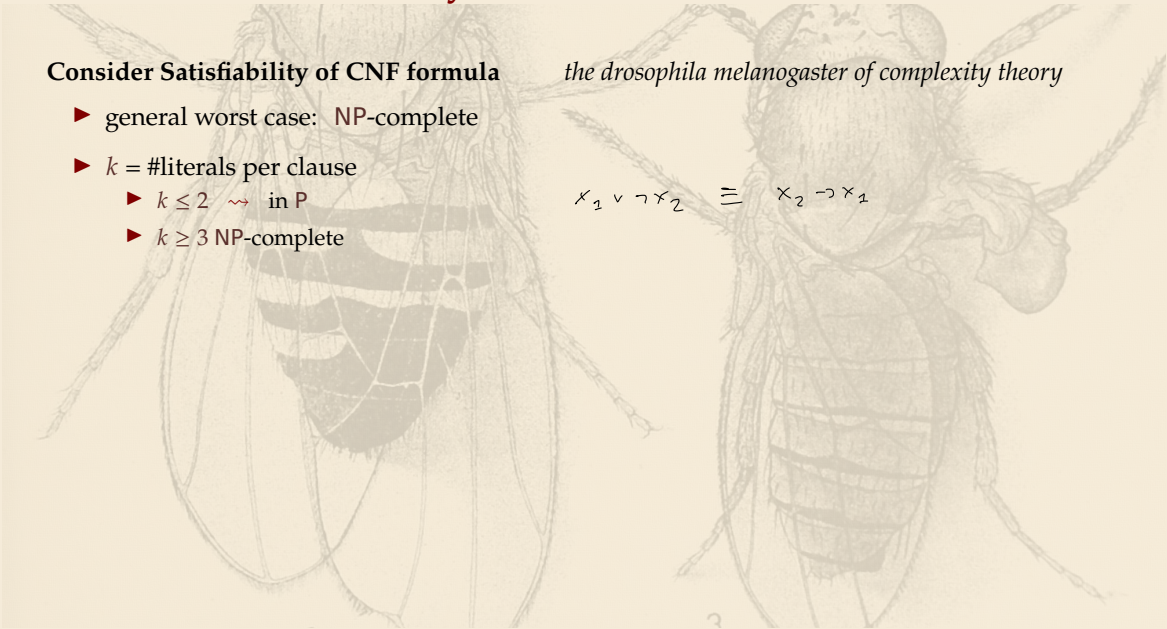
  $\rightsquigarrow$ Try to find a parameter k such that
  (1) the problem can be solved efficiently as long as k is small, and
  (2) practical instances have small values of k (even where n gets big).

Motivation: Satisfiability

Consider Satisfiability of CNF formula *the drosophila melanogaster of complexity theory*

- general worst case: NP-complete
- k = #literals per clause
  - $k \leq 2 \quad \rightsquigarrow$ in P
  - $k \geq 3$ NP-complete

$x_1 \vee \neg x_2 \quad \equiv \quad x_2 \rightarrow x_1$



Motivation: Satisfiability

Consider Satisfiability of CNF formula *the drosophila melanogaster of complexity theory*

- general worst case: NP-complete
- k = #literals per clause
  - $k \leq 2 \rightsquigarrow$ in P
  - $k \geq 3$ NP-complete
- k = #variables
  - $O(2^k \cdot n)$ time possible (try all assignments)
- k = #clauses?
- k = #literals?
- k = #ones in satisfying assignment
- k = structural property of formula
- for MAX-SAT, k = #optimal clauses to satisfy

more reasonable

Parameters

Definition 4.1 (Parameterization)

Let $\Pi = (L, Y, \Sigma)$ be a decision problem. A *parameterization* of Π is a polytime computable mapping $\kappa : L \to \mathbb{N}$. ◀

Definition 4.2 (Parameterized problem)

A *parameterized (decision) problem* is a pair (Π, κ) of a decision problem Π and a parameterization κ of Π. ◀

Examples

As before: Typically leave encoding implicit.

Definition 4.3 (p-variables-SAT)

Given: formula boolean ϕ (same as before)

Parameter: number of variables

Question: Is there a satisfying assignment $v : [n] \to \{0,1\}$? ◀

Definition 4.4 (p-Clique)

Given: graph $G = (V, E)$ and $k \in \mathbb{N}$

Parameter: k

Question: $\exists V' \subseteq V \;:\; |V'| \geq k \;\wedge\; \forall u, v \in V' : \{u, v\} \in E$? ◀

Definition 4.5 (p-Vertex-Cover)

Given: graph $G = (V, E)$ and $k \in \mathbb{N}$

Parameter: k

Question: $\exists S \subseteq V \;:\; |S| \leq k \;\wedge\; \forall e \in E : S \cap e \neq \emptyset$? ◀

Canonical Parameterization

Typical use case: study (decision versions of) optimization problems

Definition 4.6 (Canonical Parameterization)

If the instances of a decision problem Π are pairs (x, k) with $k \in \mathbb{N}$, the *canonical parameterization* is $\kappa(x, k) = k$. We write $p\text{-}\Pi$ for (Π, κ).

For $\Pi = (\Sigma_I, \Sigma_O, L, L_I, M, \mathit{value}, \mathit{goal})$ an optimization problem, $p\text{-}\Pi$ denotes the *(canonically) parameterized (decision) problem* given by the threshold problem $\mathit{Threshold}_\Pi$:
$p\text{-}\Pi = p\text{-}\mathit{Threshold}_\Pi$. ◀

Recall: $\mathit{Threshold}_\Pi$ is the set of pairs (x, k) of all instances $x \in L_I$ that have solutions that are weakly "better" than k.

Canonical Parameterization

Typical use case: study (decision versions of) optimization problems

Definition 4.6 (Canonical Parameterization)

If the instances of a decision problem Π are pairs (x, k) with $k \in \mathbb{N}$, the *canonical parameterization* is $\kappa(x, k) = k$. We write $p\text{-}\Pi$ for (Π, κ).

For $\Pi = (\Sigma_I, \Sigma_O, L, L_I, M, \mathit{value}, \mathit{goal})$ an optimization problem, $p\text{-}\Pi$ denotes the *(canonically) parameterized (decision) problem* given by the threshold problem $\mathit{Threshold}_\Pi$:
$p\text{-}\Pi = p\text{-}\mathit{Threshold}_\Pi$. ◀

Recall: $\mathit{Threshold}_\Pi$ is the set of pairs (x, k) of all instances $x \in L_I$ that have solutions that are weakly "better" than k.

Examples:

- p-Clique
- p-Vertex-Cover
- p-Graph-Coloring
- ...

Naming convention for other parameters:
p-clause-CNF-SAT: CNF-SAT with parameter "number of *clauses*"

4.1 Fixed-Parameter Tractability

Exemplary Running Times of Parameterized Problems

- *p-variables*-SAT
  - k variables, n length of formula
  - $\rightsquigarrow$ $O(2^k \cdot n)$ running time

(consider simplest brute-force methods for problems)

Exemplary Running Times of Parameterized Problems

(consider simplest brute-force methods for problems)

- p-*variables*-SAT
  - k variables, n length of formula
  - $\rightsquigarrow$ $O(2^k \cdot n)$ running time
- p-Clique
  - k threshold (clique size); n vertices, m edges in graph
  - $\rightsquigarrow$ $\binom{n}{k}$ candidates to check, each takes time $O(k^2)$ to check
  - $\rightsquigarrow$ Total time $O(n^k \cdot k^2)$

Exemplary Running Times of Parameterized Problems

- p-*variables*-SAT (consider simplest brute-force methods for problems)
  - k variables, n length of formula
  - $\rightsquigarrow$ $O(2^k \cdot n)$ running time
- p-Clique
  - k threshold (clique size); n vertices, m edges in graph
  - $\rightsquigarrow$ $\binom{n}{k}$ candidates to check, each takes time $O(k^2)$ to check
  - $\rightsquigarrow$ Total time $O(n^k \cdot k^2)$
- p-VertexCover
  - k threshold (VC size); n vertices, m edges in graph
  - $\rightsquigarrow$ $\binom{n}{k}$ candidates to check, each takes time $O(m)$ to check
  - $\rightsquigarrow$ Total time $O(n^k \cdot m)$

Exemplary Running Times of Parameterized Problems

- p-*variables*-SAT (consider simplest brute-force methods for problems)
  - k variables, n length of formula
  - $\rightsquigarrow$ $O(2^k \cdot n)$ running time

to get poly time $k = O(\log n)$ FPT

- p-CLIQUE
  - k threshold (clique size); n vertices, m edges in graph
  - $\rightsquigarrow$ $\binom{n}{k}$ candidates to check, each takes time $O(k^2)$ to check
  - $\rightsquigarrow$ Total time $O(n^k \cdot k^2)$

$k = O(1)$ for overall poly time

- p-VERTEXCOVER

XP

  - k threshold (VC size); n vertices, m edges in graph
  - $\rightsquigarrow$ $\binom{n}{k}$ candidates to check, each takes time $O(m)$ to check
  - $\rightsquigarrow$ Total time $O(n^k \cdot m)$

$k = O(1)$

- p-GRAPHCOLORING
  - k threshold (#colors); n vertices, m edges in graph
  - $\rightsquigarrow$ k^n candidates to check, each takes time $O(m)$
  - $\rightsquigarrow$ Total time $O(k^n \cdot m)$

$k = 1$

$k = 3 \Rightarrow$ NP-hard

$k = 2 \in P$ (bipartite)

FPT Running Time

Definition 4.7 (fpt-algorithm)

Let κ be a parameterization for $\Sigma^\star$.
A (deterministic) algorithm A (with input alphabet Σ) is a *fixed-parameter tractable algorithm (fpt-algorithm)* w.r.t. κ if its running time on $x \in \Sigma^\star$ with $\kappa(x) = k$ is at most

$$f(k) \cdot p(|x|) \quad = \quad O\big(f(k) \cdot |x|^c\big)$$

where p is a polynomial of degree c and f is an **arbitrary** computable function. ◀

FPT Running Time

Definition 4.7 (fpt-algorithm)

Let κ be a parameterization for $\Sigma^\star$.
A (deterministic) algorithm A (with input alphabet Σ) is a *fixed-parameter tractable algorithm (fpt-algorithm)* w.r.t. κ if its running time on $x \in \Sigma^\star$ with $\kappa(x) = k$ is at most

$$f(k) \cdot p(|x|) \quad = \quad O\big(f(k) \cdot |x|^c\big)$$

where p is a polynomial of degree c and f is an **arbitrary** computable function. ◀

Definition 4.8 (FPT)

A parameterized problem (L, κ) is *fixed-parameter tractable* if there is an fpt-algorithm that decides it.
The complexity class of all such problems is denoted by FPT. ◀

Intuitively, FPT plays the role of P.

A First FPT Example

Theorem 4.9 (p-variables-SAT is FPT)

p-variables-SAT $\in$ FPT. ◀

A First FPT Example

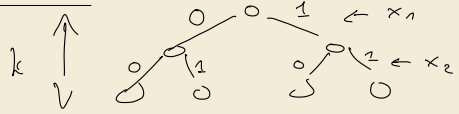
Theorem 4.9 (p-variables-SAT is FPT)

p-variables-SAT $\in$ FPT.

Proof:

Suffices to use brute force satisfiability for *p-variables*-SAT

```
1 procedure bruteForceSat(φ, X = {x_1, ..., x_k})
2     if k == 0
3         if φ == true return ∅ else UNSATISFIABLE
4     for value in {true, false} do
5         A := {x_1 ↦ value}
6         ψ := φ[x_1/value] // Substitute value for x_1
7         B := bruteForceSat(ψ, {x_2, ..., x_k})
8         if B ≠ UNSATISFIABLE
9             return A ∪ B
```



. . . but #variables not usually small

A First FPT Example

Theorem 4.9 (p-variables-SAT is FPT)

p-variables-SAT $\in$ FPT. ◀

Proof:
Suffices to use brute force satisfiability for *p-variables*-SAT

```
1 procedure bruteForceSat(φ, X = {x_1, ..., x_k})
2     if k == 0
3         if φ == true return ∅ else UNSATISFIABLE
4     for value in {true, false} do
5         A := {x_1 ↦ value}
6         ψ := φ[x_1/value] // Substitute value for x_1
7         B := bruteForceSat(ψ, {x_2, ..., x_k})
8         if B ≠ UNSATISFIABLE
9             return A ∪ B
```

Worst case running time: $O(2^k\, n)$ for $n = |\varphi|$.
2^k recursive calls;
base case needs time $O(|\phi|)$ to check whether formula evaluates to *true* ■

A First FPT Example

Theorem 4.9 (p-variables-SAT is FPT)

p-variables-SAT $\in$ FPT.

Proof:

Suffices to use brute force satisfiability for *p-variables*-SAT

```
1 procedure bruteForceSat(φ, X = {x_1, ..., x_k})
2     if k == 0
3         if φ == true return ∅ else UNSATISFIABLE
4     for value in {true, false} do
5         A := {x_1 ↦ value}
6         ψ := φ[x_1/value] // Substitute value for x_1
7         B := bruteForceSat(ψ, {x_2, ..., x_k})
8         if B ≠ UNSATISFIABLE
9             return A ∪ B
```

Worst case running time: $O(2^k\, n)$ for $n = |\varphi|$.

2^k recursive calls;

base case needs time $O(|\phi|)$ to check whether formula evaluates to *true*

. . . but #variables not usually small

Aren't we all FPT?

Theorem 4.10 (k never decreases → FPT)
Let $g : \mathbb{N} \to \mathbb{N}$ weakly increasing, unbounded and computable, and κ a parameterization with

$$\forall x \in \Sigma^\star : \kappa(x) \geq g(|x|).$$

Then $(L, \kappa) \in \mathsf{FPT}$ for *any* decidable L.

g weakly increasing: $n \leq m \;\to\; g(n) \leq g(m)$
g unbounded: $\forall t \; \exists n \; : \; g(n) \geq t$

Proof:
Y decidable $\rightsquigarrow$ $\exists$ algorithm A deciding Π in time $\leq T(|x|)$ for *some* computable $T : \mathbb{N} \to \mathbb{N}$

w.l.o.g. T is weakly increasing and $T(n) \geq n$.

Aren't we all FPT?

Theorem 4.10 (k never decreases → FPT)
Let $g : \mathbb{N} \to \mathbb{N}$ weakly increasing, unbounded and computable, and κ a parameterization with

$$\forall x \in \Sigma^\star : \kappa(x) \geq g(|x|).$$

Then $(L, \kappa) \in \mathsf{FPT}$ for *any* decidable L. ◀

g weakly increasing: $n \leq m \;\to\; g(n) \leq g(m)$
g unbounded: $\forall t \; \exists n \; : \; g(n) \geq t$

Proof:
Y decidable $\rightsquigarrow$ $\exists$ algorithm A deciding Π in time $\leq T(|x|)$ for *some* computable $T : \mathbb{N} \to \mathbb{N}$

w.l.o.g. T is weakly increasing and $T(n) \geq n$.

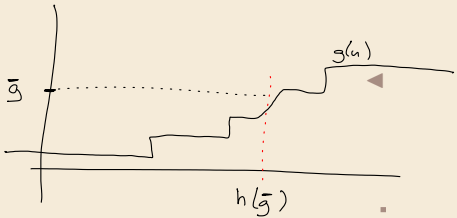
Idea: "hide" $T(|x|)$ inside $f(k)$: since $k = \kappa(x) \geq g(|x|)$ and g is unbounded, a large input forces a large parameter.

Aren't we all FPT?

Theorem 4.10 (k never decreases → FPT)

Let $g : \mathbb{N} \to \mathbb{N}$ weakly increasing, unbounded and computable, and κ a parameterization with

$$\forall x \in \Sigma^\star : \kappa(x) \geq g(|x|).$$



Then $(L, \kappa) \in \mathsf{FPT}$ for *any* decidable L.

g weakly increasing: $n \leq m \;\to\; g(n) \leq g(m)$
g unbounded: $\forall t \; \exists n \; : \; g(n) \geq t$

Proof:

Y decidable $\rightsquigarrow$ $\exists$ algorithm A deciding Π in time $\leq T(|x|)$ for *some* computable $T : \mathbb{N} \to \mathbb{N}$

w.l.o.g. T is weakly increasing and $T(n) \geq n$.

Idea: "hide" $T(|x|)$ inside $f(k)$: since $k = \kappa(x) \geq g(|x|)$ and g is unbounded, a large input forces a large parameter.

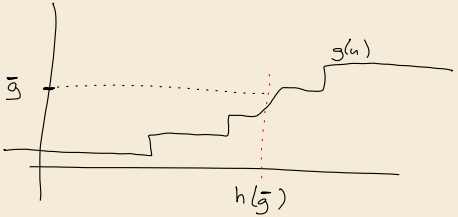
$\rightsquigarrow$ Define $h(k) \;:=\; \max\big(\{\, n \in \mathbb{N} : g(n) \leq k \,\} \cup \{1\}\big)$.

Aren't we all FPT? – Proof

Proof (cont.):

$h(k) \coloneqq \max\big(\{\, n \in \mathbb{N} : g(n) \leq k \,\} \cup \{1\}\big)$.

1. g weakly increasing and unbounded $\rightsquigarrow$ the set is finite $\rightsquigarrow$ h well-defined ✓
2. h weakly increasing
3. g computable $\rightsquigarrow$ h computable (test $n = 1, 2, \ldots$ until $g(n) > k$)
4. $h(g(n)) \geq n$ for all n (as n is in the set for $k = g(n)$)



Aren't we all FPT? – Proof

Proof (cont.):

$h(k) \;:=\; \max\big(\{\, n \in \mathbb{N} : g(n) \le k \,\} \cup \{1\}\big).$

1. g weakly increasing and unbounded $\rightsquigarrow$ the set is finite $\rightsquigarrow$ h well-defined
2. h weakly increasing
3. g computable $\rightsquigarrow$ h computable (test $n = 1, 2, \ldots$ until $g(n) > k$)
4. $h(g(n)) \ge n$ for all n (as n is in the set for $k = g(n)$)

Now let x be an input, $n = |x|$, $k = \kappa(x) \ge g(n)$. Running A on x takes time

$$T(n) \;\le\; T\big(h(g(n))\big) \;\le\; T\big(h(k)\big) \;=:\; f(k),$$

using (4) and T increasing, then $g(n) \le k$ and h, T increasing.

Aren't we all FPT? – Proof

Proof (cont.):

$h(k) \;:=\; \max\big(\{\, n \in \mathbb{N} : g(n) \le k \,\} \cup \{1\}\big).$

1. g weakly increasing and unbounded $\rightsquigarrow$ the set is finite $\rightsquigarrow$ h well-defined
2. h weakly increasing
3. g computable $\rightsquigarrow$ h computable (test $n = 1, 2, \ldots$ until $g(n) > k$)
4. $h(g(n)) \ge n$ for all n (as n is in the set for $k = g(n)$)

Now let x be an input, $n = |x|$, $k = \kappa(x) \ge g(n)$. Running A on x takes time

$$T(n) \;\le\; T\big(h(g(n))\big) \;\le\; T\big(h(k)\big) \;=:\; f(k),$$

using (4) and T increasing, then $g(n) \le k$ and h, T increasing.

f is computable, so A is an fpt-algorithm for (Π, κ) (with $p(n) \equiv 1$). ■

Back to “sensible” parameters

- ⇝ always check if parameter is reasonable (can be expected to be small)
  - if not, FPT might not even mean in NP!

Back to “sensible” parameters

- ⇝ always check if parameter is reasonable (can be expected to be small)
  - if not, FPT might not even mean in NP!
- but now, for some positive examples!

4.2 Depth-Bounded Exhaustive Search I

FPT Design Pattern

- The simplest FPT algorithms use exhaustive search
- but with a search tree bounded by $f(k)$

FPT Design Pattern

- The simplest FPT algorithms use exhaustive search
- but with a search tree bounded by $f(k)$

- bruteforceSat was a typical example!
- does this work on other problems?

Depth-Bounded Search for Vertex Cover

Let's try p-VertexCover.

Key insight: for every edge $\{v, w\}$, any vertex cover must contain v or w

Depth-Bounded Search for Vertex Cover

Let's try p-VertexCover.

Key insight: for every edge $\{v, w\}$, any vertex cover must contain v or w

```
1  procedure simpleFptVertexCover(G = (V, E), k):
2      if E == ∅ then return ∅
3      if k == 0 then return NOT_POSSIBLE  // truncate search
4      Choose {v, w} ∈ E (arbitrarily)
5      for u in {v, w} do:
6          G_u := (V \ {u}, E \ {{u, x} ∈ E})  // Remove u from G
7          C_u := simpleFptVertexCover(G_u, k − 1)
8      if C_v == NOT_POSSIBLE then return C_w ∪ {w}
9      if C_w == NOT_POSSIBLE then return C_v ∪ {v}
10     if |C_v| ≤ |C_w| then return C_v ∪ {v} else return C_w ∪ {w}
```

Depth-Bounded Search for Vertex Cover

Let's try p-VertexCover.
Key insight: for every edge $\{v, w\}$, any vertex cover must contain v or w

```
1  procedure simpleFptVertexCover(G = (V, E), k):
2      if E == ∅ then return ∅
3      if k == 0 then return NOT_POSSIBLE  // truncate search
4      Choose {v, w} ∈ E (arbitrarily)
5      for u in {v, w} do:
6          G_u := (V \ {u}, E \ {{u, x} ∈ E})  // Remove u from G
7          C_u := simpleFptVertexCover(G_u, k − 1)
8      if C_v == NOT_POSSIBLE then return C_w ∪ {w}
9      if C_w == NOT_POSSIBLE then return C_v ∪ {v}
10     if |C_v| ≤ |C_w| then return C_v ∪ {v} else return C_w ∪ {w}
```

induced subgraph of G (by subset of vertices $V \setminus \{u\}$)

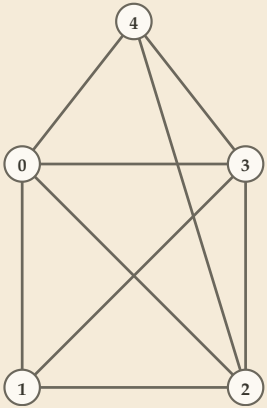
$G_u = G[V \setminus \{u\}]$

- Does not need explicit checks of solution candidates!
- runs in time $O(2^k(n+m))$ $\rightsquigarrow$ fpt-algorithm for p-Vertex-Cover

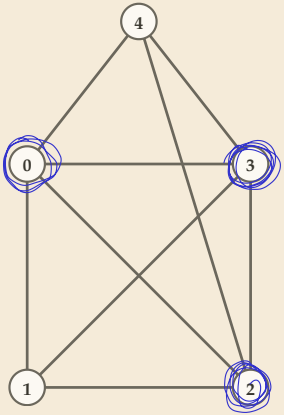
$T(m,k) \leq 2\,T(m,k-1) + \Theta(m) \quad \Rightarrow \quad T(m,k) = 2^k \cdot \Theta(m)$

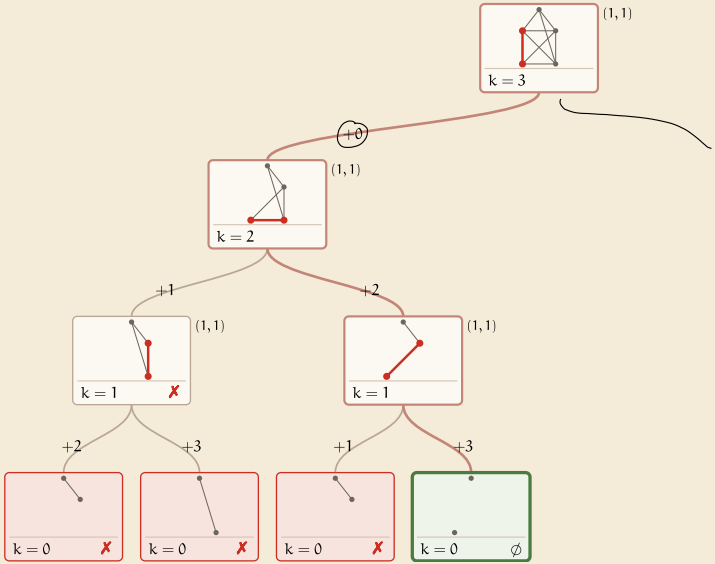
Note: $\binom{n}{k} = \Theta(n^k) \gg 2^k$

Depth-Bounded Search for Vertex Cover – Example



Depth-Bounded Search for Vertex Cover – Example





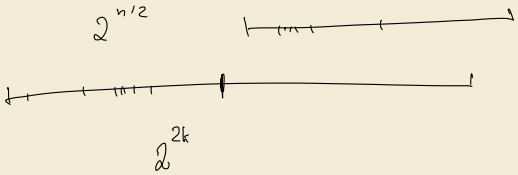
Guessing the parameter

- Note: Previous algorithm only uses k to *truncate* branches.

⇝ We can *guess* a k and it still works

binary search too expensive !

exponential search



linear search 2^k

Guessing the parameter

- Note: Previous algorithm only uses k to *truncate* branches.

$\rightsquigarrow$ We can *guess* a k and it still works

$\rightsquigarrow$ Try all k!

```
1 procedure vertexCoverBfs(G = (V, E))
2     for k := 0, 1, ..., |V| do
3         C := simpleFptVertexCover(G, k)
4         if C ≠ NOT_POSSIBLE return C
```

Guessing the parameter

- Note: Previous algorithm only uses k to *truncate* branches.

⇝ We can *guess* a k and it still works

⇝ Try all k!

```
1 procedure vertexCoverBfs(G = (V, E))
2     for k := 0, 1, ..., |V| do
3         C := simpleFptVertexCover(G, k)
4         if C ≠ NOT_POSSIBLE return C
```

- Running time: $\sum_{k'=0}^{k} O(2^{k'}(n+m)) \;=\; O(2^{k}(n+m))$

⇝ For exponentially growing cost, trying all values up to k costs only constant factor more

4.3 Problem Kernels

Preprocessing

- Second key fpt technique are *reduction rules*
- **Idea:** Reduce the size of the instance (in polytime) without changing its outcome



Preprocessing

- Second key fpt technique are *reduction rules*
- **Idea:** Reduce the size of the instance (in polytime) without changing its outcome
- Trivial example for SAT:

  If a CNF formula contains a single-literal clause $\{x\}$ resp. $\{\neg x\}$, set x to *true* resp. *false* and remove the clause.

  - doesn't do anything in the worst case ...

Preprocessing

- Second key fpt technique are *reduction rules*
- **Idea:** Reduce the size of the instance (in polytime) without changing its outcome
- Trivial example for SAT:

  If a CNF formula contains a single-literal clause $\{x\}$ resp. $\{\neg x\}$, set x to *true* resp. *false* and remove the clause.

  - doesn't do anything in the worst case ...
  - special case of resolution calculus rule $\dfrac{a_1 \vee a_2 \vee \cdots \vee x, \quad b_1 \vee b_2 \vee \cdots \vee \neg x}{a_1 \vee a_2 \vee \cdots \vee b_1 \vee b_2 \vee \cdots}$
  - basis of practical SAT solvers

Preprocessing

- Second key fpt technique are *reduction rules*
- **Idea:** Reduce the size of the instance (in polytime) without changing its outcome
- Trivial example for SAT:

  If a CNF formula contains a single-literal clause $\{x\}$ resp. $\{\neg x\}$, set x to *true* resp. *false* and remove the clause.

  - doesn't do anything in the worst case ...
  - special case of resolution calculus rule $\dfrac{a_1 \vee a_2 \vee \cdots \vee x, \quad b_1 \vee b_2 \vee \cdots \vee \neg x}{a_1 \vee a_2 \vee \cdots \vee b_1 \vee b_2 \vee \cdots}$
  - basis of practical SAT solvers
- Trivial example for VertexCover

  Remove vertices of degree 0 or 1.

  include other endpoint in solution

  (never needed as part of optimal VC)

Preprocessing

- Second key fpt technique are *reduction rules*
- **Idea:** Reduce the size of the instance (in polytime) without changing its outcome
- Trivial example for SAT:

  If a CNF formula contains a single-literal clause $\{x\}$ resp. $\{\neg x\}$, set x to *true* resp. *false* and remove the clause.

  - doesn't do anything in the worst case ...
  - special case of resolution calculus rule $\dfrac{a_1 \vee a_2 \vee \cdots \vee x, \quad b_1 \vee b_2 \vee \cdots \vee \neg x}{a_1 \vee a_2 \vee \cdots \vee b_1 \vee b_2 \vee \cdots}$
  - basis of practical SAT solvers
- Trivial example for VertexCover

  Remove vertices of degree 0 or 1.

  include other endpoint in solution

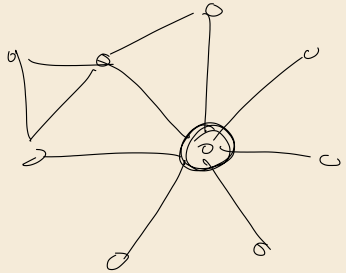
  (never needed as part of optimal VC)
- Here: reduction rules that provably shrink an instance to size $g(k)$

Buss's Reduction Rule for VC

- Given a p-VertexCover instance (G, k)

  "deg > k" Rule: If G contains vertex v of degree $\deg(v) > k$, include v in potential solution and remove it from the graph.

- Can apply this simultaneously to degree $> k$ vertices.
- Either rule applies, or all vertices bounded degree(!)



Kernels

Definition 4.11 (Kernelization)

preprocessing
=

Let (L, κ) be a parameterized problem. A function $K : \Sigma^\star \to \Sigma^\star$ is *kernelization* of L w.r.t. κ if it maps any $x \in L$ to an instance $x' = K(x)$ with ~~$k' \leq \kappa(x')$~~ so that

1. (self-reduction) $x \in L \iff x' \in L$
2. (polytime) K is computable in polytime.
3. (kernel-size) $|x'| \leq g(k)$ for some computable function g

We call x' the *(problem) kernel* of x and g the *size of the problem kernel*. ◀

Buss's Kernel

Buss's Reduction for Vertex Cover: (repeatedly apply until no more changes)

- $\deg > k$ rule
- Remove degree 0 and 1 vertices

Theorem 4.12 (Buss's Reduction is Kernelization)
Buss' reduction yields a kernelization for p-Vertex-Cover with kernel size $O(k^2)$. ◀

Buss's Kernel

Buss's Reduction for Vertex Cover: (repeatedly apply until no more changes)

- $\deg > k$ rule
- Remove degree 0 and 1 vertices

Theorem 4.12 (Buss's Reduction is Kernelization)

Buss' reduction yields a kernelization for p-Vertex-Cover with kernel size $O(k^2)$. ◀

Proof:

After repeatedly applying Buss's rule as well as the isolated/leaf rule until neither applies further, we have $\forall v \in V : 2 \leq \deg(v) \leq k$.

(Note that the rule might reduce the parameter k).

Buss's Kernel

Buss's Reduction for Vertex Cover: (repeatedly apply until no more changes)

- $\deg > k$ rule
- Remove degree 0 and 1 vertices

Theorem 4.12 (Buss's Reduction is Kernelization)
Buss' reduction yields a kernelization for p-Vertex-Cover with kernel size $O(k^2)$. ◀

Proof:
After repeatedly applying Buss's rule as well as the isolated/leaf rule until neither applies further, we have $\forall v \in V : 2 \leq \deg(v) \leq k$.
(Note that the rule might reduce the parameter k).
In the resulting graph, any VC of size $\leq k$ covers $\leq k^2$ edges.

Buss's Kernel

Buss's Reduction for Vertex Cover: (repeatedly apply until no more changes)

- $\deg > k$ rule
- Remove degree 0 and 1 vertices

Theorem 4.12 (Buss's Reduction is Kernelization)

Buss' reduction yields a kernelization for p-Vertex-Cover with kernel size $O(k^2)$. ◀

Proof:

After repeatedly applying Buss's rule as well as the isolated/leaf rule until neither applies further, we have $\forall v \in V : 2 \leq \deg(v) \leq k$.
(Note that the rule might reduce the parameter k).
In the resulting graph, any VC of size $\leq k$ covers $\leq k^2$ edges.
If $m > k^2$, we output a trivial No-instance (e. g., a K_{k+1} a complete graph on $k+1$ vertices).

Buss's Kernel

Buss's Reduction for Vertex Cover: (repeatedly apply until no more changes)

- $\deg > k$ rule
- Remove degree 0 and 1 vertices

Theorem 4.12 (Buss's Reduction is Kernelization)
Buss' reduction yields a kernelization for p-Vertex-Cover with kernel size $O(k^2)$. ◀

Proof:
After repeatedly applying Buss's rule as well as the isolated/leaf rule until neither applies further, we have $\forall v \in V : 2 \leq \deg(v) \leq k$.



(Note that the rule might reduce the parameter k).
In the resulting graph, any VC of size $\leq k$ covers $\leq k^2$ edges.
If $m > k^2$, we output a trivial No-instance (e. g., a K_{k+1} a complete graph on $k+1$ vertices).
If $m \leq k^2$, then the input size is now bounded by $g(k) = 2k^2$. ■

FPT iff Kernelization

Theorem 4.13 (FPT ↔ kernel)
A computable, parameterized problem (Π, κ) is fixed-parameter tractable if and only if there is a kernelization for L w.r.t. κ.

Proof:

"⇐" kernelization K for (Π, κ) given. $\forall: x \in L \quad x \in Y \Leftrightarrow K(x) \in Y$

$|K(x)| \leq g(\underbrace{\kappa(x)}_{k})$

Y has algorithm A deciding $x \in Y$ in running time $T(n)$, wlog T weakly increasing

Construct fpt-algorithm for (Π, κ)

(1) given $x \in L$, compute $x' := K(x)$ in poly time

$|x'| \leq g(k)$

(2) run A on x'

time $T(|x'|) \leq T(g(k)) =: f(k)$

⇒ total time for deciding $x \in Y$ is $\mathcal{O}(f(k) + n^c)$ for constant c

FPT iff Kernelization [2]

Proof (cont.):

"$\Rightarrow$" Given fpt-algorithm A for (Π, κ) with time $\leq f(k) \cdot n^c$

(1) Simulate A for $\leq n^{c+1}$ steps (poly time)

(2) If A terminates before n^{c+1} steps

if answer from A was "yes": output trivial yes-instance

if answer from A was "no": output trivial no-instance

Otherwise $n^{c+1} \leq f(k) \cdot n^c \Rightarrow n \leq f(k)$

output original input.

Max-SAT Kernel

Theorem 4.14 (Kernel for Max-SAT)

p-MAX-SAT has a problem kernel of size $O(k^2)$ which can be constructed in linear time.

Proof: Given formula φ with m clauses, can we satisfy $\geq k$ clauses

wlog. every variable occurs at most once per clause

($x \vee \neg x \in C \rightsquigarrow$ drop C, always satisfied)

n = total # literals in all clauses (try to make $n = O(k^2)$)

Case 1 $k \leq \lfloor \frac{m}{2} \rfloor$ Output true

Pick arbitrary assignment of all variables

$\Rightarrow \ell$ clauses satisfied if $\ell \geq k$ ✓

otherwise $\ell < k \leq \lfloor \frac{m}{2} \rfloor$ ($\Leftrightarrow \ell \leq \frac{m}{2}$)

negating all variables $\Rightarrow m - \ell \geq m - \lfloor \frac{m}{2} \rfloor \geq \lfloor \frac{m}{2} \rfloor \geq k$

clauses satisfied

Max-SAT Kernel [2]

Proof (cont.):

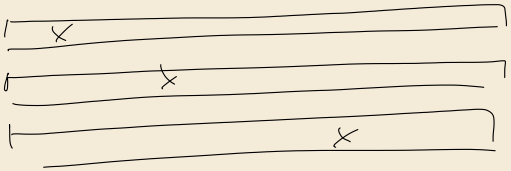
Case 2: $k > \lfloor \frac{m}{2} \rfloor \Rightarrow k > \frac{m}{2} \Rightarrow m < 2k$

(few clauses but n could still be large)

$F_L := \{$ clauses C : C has $\geq k$ literals $\}$ $\quad L := |F_L|$

$F_S := \{$ —— $<k$. $\}$

2a: $L \geq k$



$\exists$ disjoint set of k variables that satisfies k long clauses

$\Rightarrow$ return true

Max-SAT Kernel [3]

Proof (cont.):

2b $L < k$ Claim: $(F_S, k-L)$ is Yes-instance $\Leftrightarrow$ (F, k) is —

Corollary 4.15

p-Max-SAT $\in$ FPT